Before Fractal Continual Learning:
An Implementation Audit and a Pre-Registered Decisive Comparison

HUSHFUSION Forge Research

October 2026

"Violence is the last refuge of the incompetent."
— Salvor Hardin, Isaac Asimov, Foundation (1951)

Abstract

Continual learning is usually posed as parameter adaptation under a fixed computational graph; whether the graph itself should grow is a separate question, often asserted but rarely audited. We construct a recursive fractal-loop learner that mixes root and descendant outputs through a normalized convex gate and spawns descendants when improvement slows, and run it paired against a fixed single-node learner in a resampled-input control environment. A post-review implementation audit reveals that in the original 72,335-round run the descendant nodes were never trained—they appear in no optimizer's parameter table, and the advertised depth-dependent sparse schedule never enters a scheduling decision—so the apparent boundary result of "no structural advantage" was not a test of a fractal learner. We repair the implementation, match gradient budgets, and pre-register the decisive comparison. Here we show that under the pre-registered rule the repaired re-run (200,000 rounds, seed 20261012) is equivalent: the paired 95% bootstrap confidence interval on the warm-cost difference lies entirely within the ± 1-step equivalence bound ($n = 5{,}000$, CI $[-0.651, -0.548]$). Long-horizon adaptation is confirmed for both learners (warm cost 130→9 steps versus a random baseline of $\sim$32) and growth is operational (1→7 nodes), but in this deterministic small-model regime structural growth confers no measurable advantage under matched compute.

1 Introduction

Continual learning is normally stated as a weight-update problem: keep the graph fixed and adapt parameters so that new tasks are learned without forgetting old ones [1, 3, 2]. This framing leaves an unasked question: when the task itself changes, should the *organization of the updating loops* change, not just the weights inside them? For long-running controllers that must track a moving optimum for tens of thousands of rounds, the answer matters; yet whether structural growth helps is nearly always asserted, never measured.

We treat the question as a measurement problem. The main experiment ran a recursive fractal-loop learner for 72,335 paired rounds against a fixed single-node learner in a resampled-input control environment. A post-review audit found that the fractal tree's descendant nodes were never trained and its advertised sparse schedule never executed: the long run tested a root node gating untrained random subnets, not a fractal learner. After repairing the implementation and matching gradient budgets (Forge commit `53e4938`), we pre-registered the decision rule and launched a 200,000-round seeded re-run. The pre-registered verdict is equivalence: the paired bootstrap confidence interval lies entirely inside the ± 1-step bound, so under matched compute structural growth shows no advantage in this regime. Related work on meta-learning [1, 3], continual consolidation [2], and

Table 1: Pre-registered decisive re-run (200,000 rounds, seed 20261012, matched compute). Warm rows are recommendations; cold rows are random controls. CI is the paired bootstrap 95% interval at the pre-registered first look (n = 5,000 warm rounds, seed-fixed bootstrap). Positive difference means fractal is better (fewer steps). Verdict: *equivalent*—the CI lies entirely inside the ±1-step equivalence bound.

Metric	Value
Paired warm-cost mean diff (fixed − fractal), n=5,000	−0.60 steps
95% bootstrap CI	[−0.651, −0.548]
Equivalence bound	[−1, +1] steps
Converged-window mean diff, n=99,000	−0.15 steps
Warm mean, converged (fixed / fractal)	8.96 / 9.11
Cold mean (fixed / fractal)	31.44 / 31.53
Exceedance rate (fixed / fractal)	0.10% / 0.00%
Final tree nodes	7 (cap)

fractal allocation [6, 5] is summarized in Section 4; what stands, what is retracted, and what is now decided are the paper's full claim set (Table **??**).

2 Results

2.1 The repaired, pre-registered re-run decides: equivalence under matched compute

The audit left structural advantage undecidable (Section 2.2); the decisive test required three fixes agreed with reviewers before it ran. First, compute had to be matched: a seven-node tree consumes several times more gradient steps per round than a single node, so a naive re-run would confound structure with compute. The repaired implementation counts total gradient steps per round in the tree and gives the fixed learner the same budget (Forge commit `53e4938`). Second, the decision rule had to be fixed before the re-run started: equivalence interval ±1 step (the integer resolution of the metric), converged window round ≥ 2000, minimum 5,000 and maximum 10,000 warm rounds with a single look at 5,000, and a paired bootstrap 95% confidence interval as the verdict (details in `PREREGISTRATION.md`). Third, the run had to be reproducible: a fixed seed (20261012) and a committed trace.

The re-run completed 200,000 rounds (100,000 warm, 100,000 cold; warm rounds ≥ 2000: n = 99,000). At the pre-registered first look (n = 5,000), the paired difference in warm cost (fixed − fractal) had mean −0.60 steps with 95% bootstrap CI $[-0.651, -0.548]$—entirely inside $(-1, +1)$, the resolution of the metric (Table 1). Under the pre-registered rule this is the verdict *equivalent*: no structural advantage is measurable at or above the one-step resolution of the cost. The full converged window confirms the look (n = 99,000, mean −0.15), and the four time-segment means (−0.16, −0.12, −0.18, −0.14) show the difference is stationary, not an early transient. Safety exceedance stayed below the pre-registered 10% ceiling (fixed 0.10%, fractal 0.00%), cold baselines matched the expected ∼32 steps (31.44 / 31.53), and the fractal tree grew from 1 to 7 nodes (the cap) in this run as well.

2.2 The audit: descendants were never trained

An earlier version of this paper claimed a boundary result: "no structural advantage." A post-review audit of the main experiment's implementation retracts that claim as undecidable from that run, because three findings show the run did not test a fractal learner.

Descendants are never trained. In `design_daemon_fractal.py` (audited snapshot: commit `10ba961^`), each node's optimizer is constructed over the node's own network and gate parameters only ("opt = Adam(list(self.net.parameters()) + [self.gate])"). Children of a grown node are appended to `self.children`, but never join any parent's optimizer parameter list, and the main loop calls `train_step` only on the root. A descendant's parameters therefore never receive a gradient step; the only per-child operation is a warm-up counter decrement. The grown "tree" is a root network gating fixed, randomly initialized subnets.

The advertised sparse schedule never executes. The code computes `update_freq = D**depth` and stores it, but no scheduling decision anywhere consults it. Depth-dependent update frequency is documentation, not behavior.

The evaluation is not reproducible as stated. The 72,335-round trace and its analysis script are not in the repository; the daemon seeds its RNG from wall-clock time (`int(time.time()) % 10000`); a rerun does not reproduce the reported numbers.

Consequence: the paired long-run difference (11.5 vs. 11.1 warm steps) and the three-seed and 701-record comparisons all compare a trained root against a root gating untrained subnets—a condition that neither confirms nor refutes a fractal advantage. Long-horizon adaptation and growth detection remain valid (Sections 2.3–2.4); structural advantage was undecidable until the re-run of Section **??**.

2.3 Long-horizon adaptation is real in both root learners

Both roots learned and kept learning. Warm performance fell from roughly 130 steps in the first 1,000 rounds to roughly 9 steps after 2,000 rounds, and stayed there for the remaining 70,000 rounds; the cold random baseline remained near 32 steps throughout (Table 1). Continual adaptation was therefore effective and stable, and it did not decay. This result is independent of the descendant-training flaw: it is a property of the root learner, and both roots exhibit it.

2.4 Growth is operational but cap-stopped

The fractal tree grew six times (rounds 46, 57, 73, 80, 87, 94), from one to seven nodes, and then stopped. Growth was driven by a relative-decay detector (Methods): a descendant spawns when the exponentially weighted improvement rate of validation loss falls below a fraction of that structure's early improvement rate. After reaching seven nodes the detector never fired again for the remaining 72,000 rounds. We do not attribute this to autonomous saturation: seven is a hard safety cap in the implementation, so growth was stopped by the cap, not by an emergent equilibrium. Whether the tree would have continued growing without the cap is untested. The audit sharpens the reading: the detector and the graph mutation work over a long horizon, but in the audited run the grown components were never exercised as learners.

3 Discussion

Three statements must be held apart, and this paper now settles two of them. "Recursive structural plasticity is implementable and stable" is qualified: growth events fire and the graph mutates, but in

the audited run the grown components never learned, so we can only say the *detector and mutation* work, not the grown learner. "Structural growth improves performance" was the paper's earlier claim and is now decided in the negative direction: under matched compute, in this deterministic control environment, the pre-registered comparison is equivalent—structural growth confers no advantage resolvable at the one-step resolution of the metric. Conflating "the tree grew" with "the grown components learned" was the error the audit corrected; separating them is the paper's methodological point.

The equivalence verdict is a boundary result, not a license to generalize. It holds for a small model, a deterministic resampled-input environment, and a matched gradient budget. The prefix-concentration argument that motivates fractal allocation [5] requires marginal returns to decline sharply with depth; on a small learner with limited data they may not, and splitting the budget across specialists may dilute each specialist's evidence. Whether fractal organization pays in larger models, non-stationary environments, or unconstrained budgets remains open—and the pre-registered protocol of this paper is the template by which those claims should be tested, not asserted.

The practical lesson is procedural. Evaluate structural growth in three stages: (i) does the graph mutate safely, (ii) does the learner keep adapting after mutation, and (iii) does the mutation improve a pre-registered metric under matched compute and data? This paper ran all three; stage (iii) required repairing the implementation and pre-registering the rule, and its answer in this regime is no measurable advantage.

4 Related Work

Our design sits at the meeting point of optimization-based meta-learning [1, 3], continual-learning consolidation [2], and fractal allocation arguments rooted in Murray's minimum-work analysis of branching networks [6] and formalized in our companion Lean development [5]. We keep these as motives, not predictions; the paper's contribution is methodological—how an apparently clean long-horizon comparison can silently fail to test its hypothesis, and what a decisive test requires—rather than a claim that fractal organization either helps or hurts.

5 Methods

5.1 Recursive fractal-loop architecture

Each node v holds a predictor $f_v(x;\theta_v)$ and may have descendants $\mathcal{C}(v)$. The node output is a convex combination,

$$y_v(x) = g_{v,0}\, f_v(x;\theta_v) + \sum_{u\in\mathcal{C}(v)} g_{v,u}\, y_u(x), \qquad g_{v,j} \geq 0, \quad \sum_j g_{v,j} = 1, \tag{1}$$

with gates produced by a softmax. Descendants are not trusted immediately: they are warmed independently before entering the parent's mixture, which prevents the "dead-child" failure where a zero-initialized gate gives a child zero contribution and therefore zero gradient. The root is updated every round; a node at depth d updates at frequency D^{-d} for sparse factor D (we use $D = 4$). As the audit shows, this sparse schedule was specified but not enforced by the deployed code.

5.2 Growth rule

Growth is triggered by relative decay of validation loss. Let $\bar{\imath}_t$ be an exponentially weighted improvement rate and $\imath_0$ the structure's early average improvement rate. A descendant is spawned when $\bar{\imath}_t/\imath_0$ stays below a threshold (0.25 in the deployed run) for several consecutive observations; the detector resets after growth. A clean-loss filter excludes safety-penalty values, because a penalty spike is not evidence of a structural bottleneck.

5.3 Control environment

We assemble a deterministic μ-dynamics controller from the two-flow-ring model:

$$\mu_{t+1} = \mu_t + \eta_t(1 - \mu_t), \qquad \eta_t = \eta_{\text{ext}} + (2\lambda - \lambda^2)(1 - \mu_t), \tag{2}$$

where η_{ext} is an external drive and λ a convergence parameter. The task is to reach $\mu \geq 0.999$ (work window) without crossing $\mu_{\text{ceil}} = 1 - m_e/m_i \approx 0.999780568$ (safety ceiling). Cost is the number of steps to the window; ceiling crossing is a failure. The environment is deterministic, has a clear safety boundary, and is resampled: a fresh $\mu_0 \sim U(0.01, 0.8)$ is drawn every round. We describe this as resampling, not drift: the mapping from control point to cost is fixed, so the learner faces input resampling rather than a changing task. It is a benchmark for method study, not evidence about any experimental antigravity device; λ remains an input gap in the upstream model.

5.4 Paired long-horizon protocol

Every round, both learners see the same μ_0. Each selects a control point from its experience (UCB over a 20×10 grid of $(\eta_{\text{ext}}, \lambda)$), executes the environment, and updates its parameters from a sliding window of recent experience (capacity 400). Odd rounds are cold random controls, even rounds are warm recommendations. The deployed run used 72,335 rounds with a supervisor that restarted the daemon on failure; the decisive re-run used 200,000 rounds, seed 20261012, and the matched-compute repair. As Section 2.2 details, the deployed implementation did not include descendant parameters in any optimizer, so the protocol description is the intended design; the decisive re-run executes the repaired design.

5.5 Matched-compute repair and pre-registered decision rule

The repair (Forge commit `53e4938`) makes each `train_step` return its total gradient-step count; each round, the fixed learner is given the same number of gradient steps the fractal tree used, so structure and compute are no longer confounded. The decision rule was fixed before the re-run started (`PREREGISTRATION.md`): paired warm-cost difference (fixed − fractal) over warm rounds ≥ 2000; equivalence interval ± 1 step; paired bootstrap 95% CI (10,000 resamples, seed fixed); verdicts equivalent / fractal-better / fixed-better / inconclusive, with one look at $n = 5{,}000$, a stop at $n = 10{,}000$ if still inconclusive, and no further looks (to prevent continuous peeking).

5.6 Failure analyses that shaped the design

Four failures are worth recording. (i) An absolute "no-improvement" growth detector never fires on shallow networks (they improve slowly forever); the relative-decay detector fixed this. (ii) Zero-initialized child gates produce dead children; uniform gate initialization plus a warm-up phase fixed this. (iii) Growing a layer without reinitializing per-parameter step-size meta-parameters damaged the fast-adaptation loop, so structural growth and meta-parameter growth cannot be

treated as independent. (iv) A standard RSI indicator is inapplicable to a monotonically improving loss trajectory (one-sided deltas give RSI $\equiv$ 100), so the term is used here for "recursive self-improvement," not the financial indicator.

Data and code availability

The audited implementation, the repair, and the analysis scripts are in the HUSHFUSION Forge repository [4]. The audited snapshot is commit `10ba961^` (children untrained); the repaired, matched-compute version is commit `53e4938`. The decisive re-run trace is `oml_daemon_fractal_matched_20261012.jsonl` (200,000 rounds, seed 20261012; 36,151,557 bytes; md5 `edc0f8e78fb0527d29d151222473f26b`; last write 2026-10-12T03:11 UTC), analyzed by `paper/analyze_matched.py`; the pre-registration, decision rule, and verdict are in `PREREGISTRATION.md`. Writing standards are in `WRITING-STANDARD.md`.

References

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[2] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. Meta-SGD: Learning to learn quickly for few-shot learning. *arXiv preprint arXiv:1707.09835*, 2017.

[4] Yuanjie Liu. HUSHFUSION Forge: design search, knowledge reuse, and continual-learning daemons. `https://github.com/logos-42/hushfusion-forge`, 2026. Long-horizon fracta-loop trace and reproduce scripts.

[5] Yuanjie Liu. LmPrinciple: Fractal allocation theorems for diminishing-return structures. `https://github.com/logos-42/lm-principle`, 2026. Lean formalization: Fractal.lean, prefix_allocation_optimal, fractal_beats_uniform.

[6] Cecil D. Murray. The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *Proceedings of the National Academy of Sciences*, 12(3):207–214, 1926.